

How this application actually works

A plain-language tour of the CAD kernel, the catalog, the 3D editor, the AI agent, saving, sharing, and the services that host it — with flowcharts for the paths a person can follow.



CONTENTS

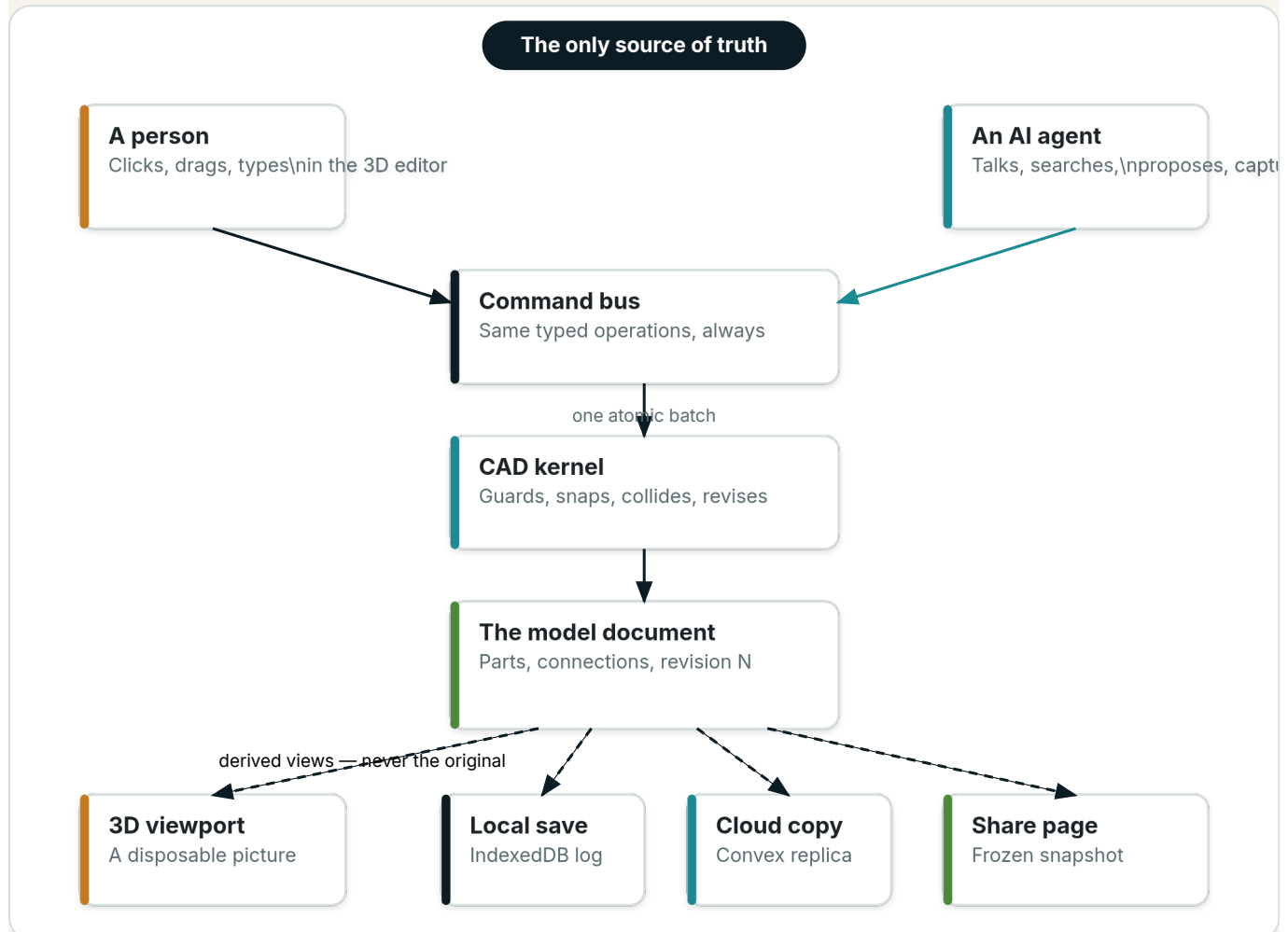
What is in this guide

1. The idea in one page
2. A walk through the product
3. Placing a brick
4. The kernel and revisions
5. The catalog of real parts
6. How bricks clutch together
7. Checking the model
8. Building walls and buildings
9. Saving locally and in the cloud
10. Accounts (Hexclave)
11. The AI assistant
12. Generating from a sentence
13. Refining a region
14. Sharing a snapshot
15. How production is wired
16. Where the code lives

One model. Two operators. Nothing is a sketch.

Brickwright is a 3D CAD program for physically buildable brick models. It is not a chat window with a picture beside it. A person and an AI agent operate the *same* revised document, the same part catalog, the same snap solver, the same undo stack, and the same 3D view.

If you remember only one sentence: **the model document is the only source of truth**. React, Three.js, chat transcripts, and cloud rows are derived views. If those views disagree with the document, the document wins and the views are rebuilt.



Everything you see is a view of one document. Humans and the agent edit it the same way.

81,774

searchable part identities in
catalog 2026-07

900

placeable parts with compiled
geometry in this build

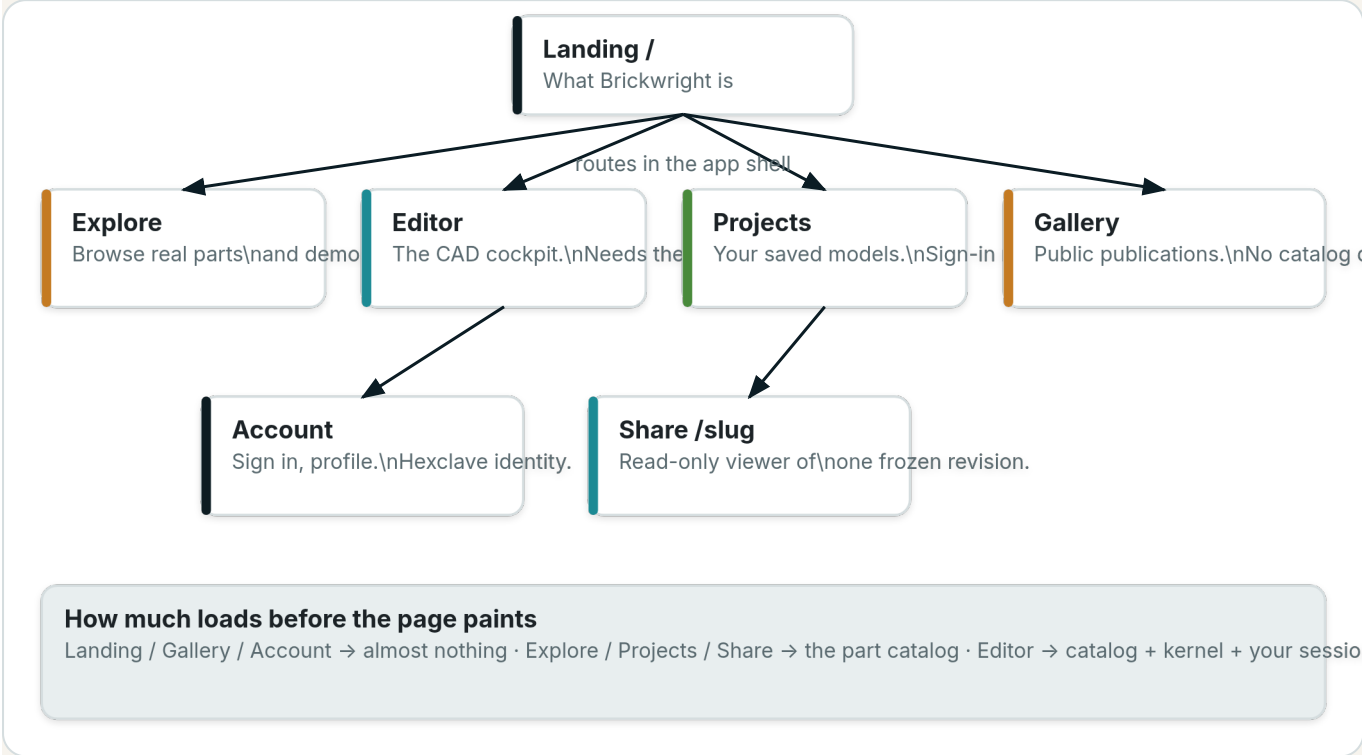
324,331

normalized connectors the snap
solver can use

Why this matters. A language model is excellent at talking and terrible at secretly inventing a stud pitch. Brickwright lets it talk, then forces every brick through the same physics the mouse already uses. If a part cannot be placed, both of you get a teaching error rather than a fake brick.

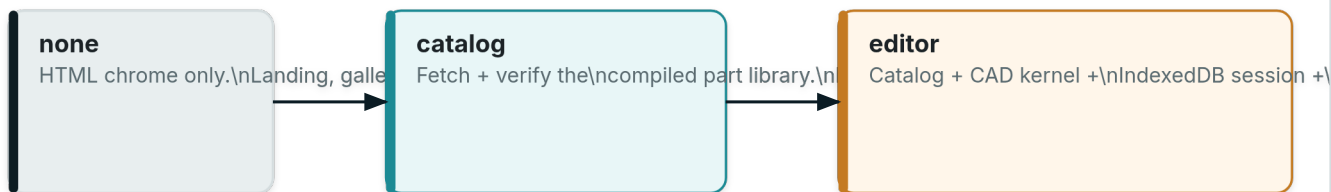
What you can open, and what each page is allowed to load

The application is a single website with a few routes. The shell decides *how much CAD* a route may download before it paints. That is why the home page is a web page and the editor is a cockpit, not why two different apps were glued together.



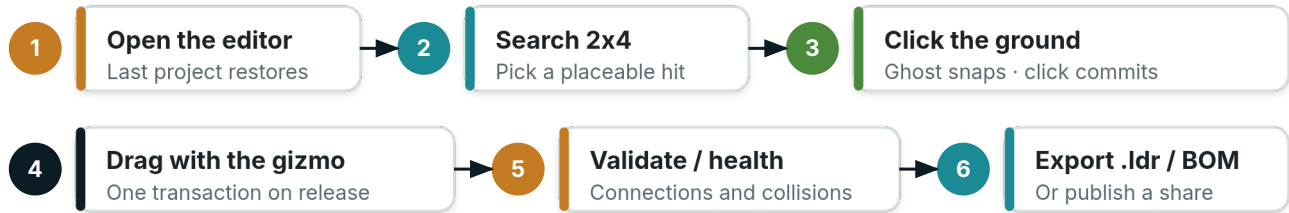
Each page only downloads what it needs. Opening the marketing site never loads a LEGO catalog.

PLACE	WHO IT IS FOR	WHAT IS TRUE THERE
/ Landing	A visitor	Explains the product. Does not fetch the brick library.
/explore	Someone browsing	Names real parts. Needs the catalog to say whether a part is real.
/editor	A builder or an agent	The only surface that mutates a document.
/projects	A signed-in person	Cloud and local projects. Needs an account.
/share/...	Anyone with the link	A frozen, read-only publication. Cannot edit the original.
/gallery	The public	Published models. Still no CAD kernel.
/account	You	Sign-in, profile. Hexclave, not the CAD document.



A route cannot promote itself. That is what keeps the home page light.

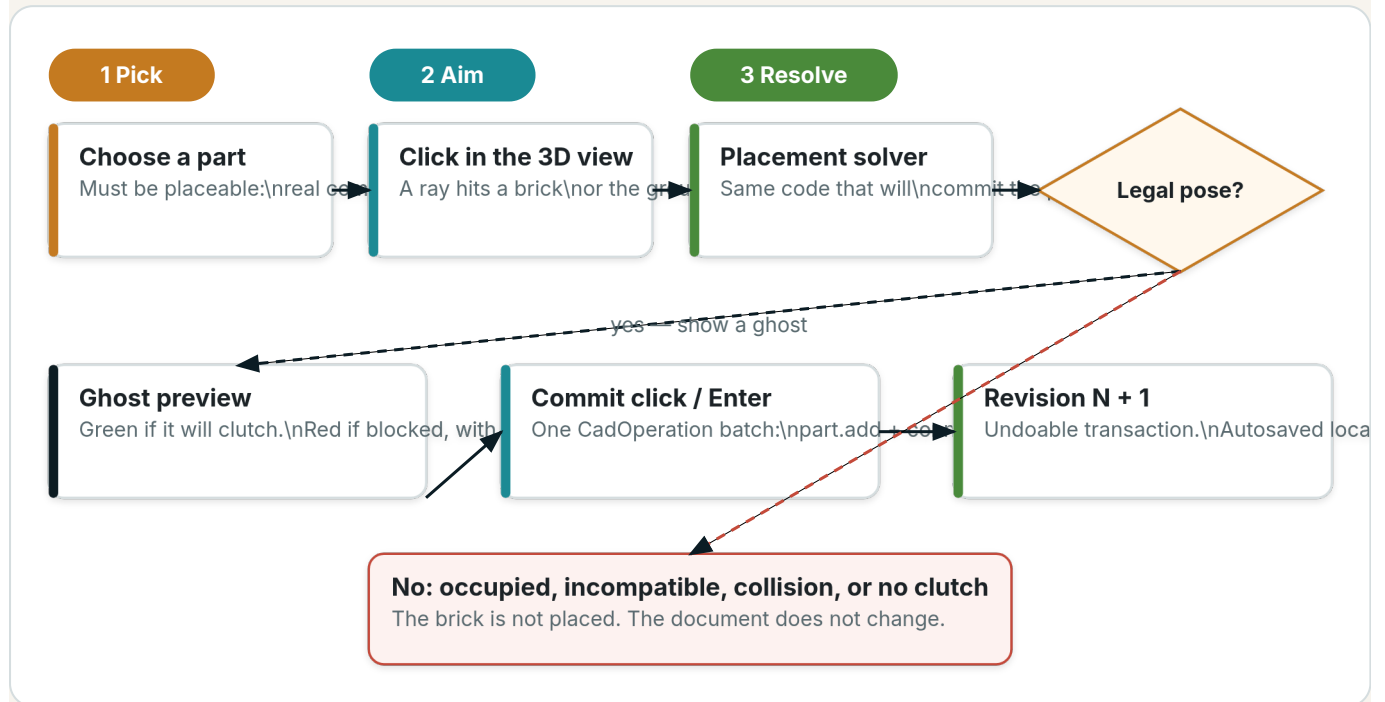
Boot is staged. If a page declared it needs the catalog and the files are missing, it refuses rather than inventing parts.



A first session without an account: pick, place, move, check, export. Cloud is optional.

What happens when you put a brick down

From the chair, it feels like a game: pick a 2×4, hover, click. Underneath, the viewport is only supplying a ray — a line from the camera through the pixel you clicked. Everything after that is kernel work, so the ghost you see is the pose that will actually be committed.



Placing a brick is not a graphics trick. The ghost you see is the kernel's real answer.

The viewport is a picture

Three.js draws instanced meshes, a studio lighting environment, a transform gizmo, and red/green ghosts. None of that is stored. The scene graph can be thrown away and rebuilt from the document at any time.

Coordinates are LDraw's

Units are LDU. Y points down. One stud is 20 LDU; a plate is 8 LDU tall; a brick is 24. Orientation is a 3×3 matrix, not yaw/pitch/roll, so a mirrored piece round-trips into .ldr exactly.

Useful keys

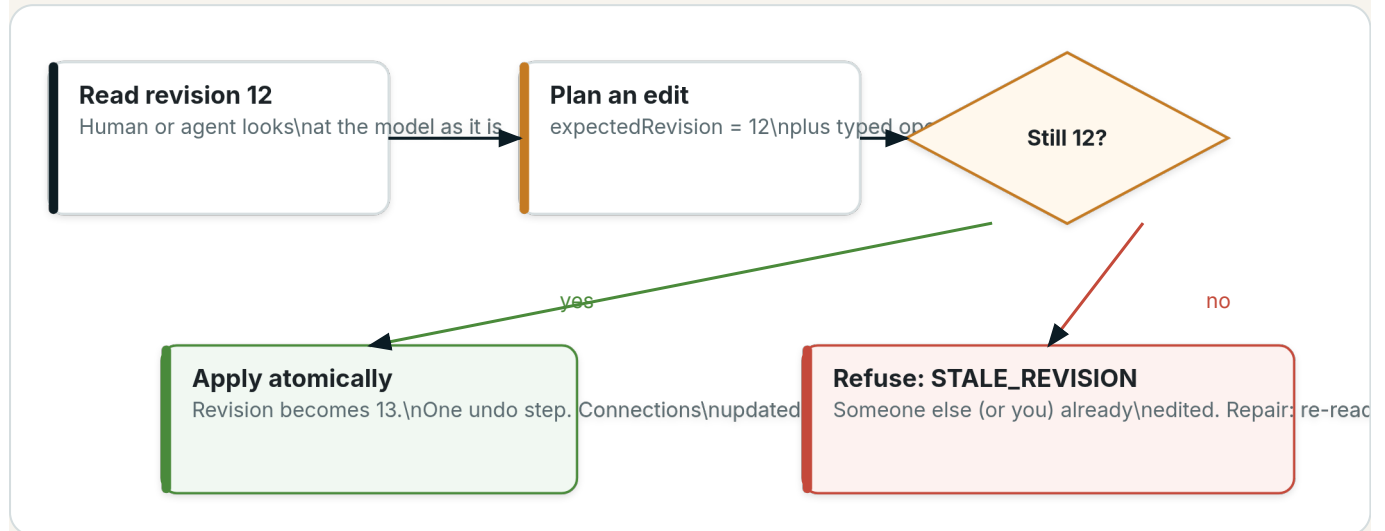
R turns the held brick. **M** picks it up to reset. **G** is the move gizmo. Shift-drag box-selects. ⌘Z / Ctrl+Z undoes the whole transaction, even if it placed a building.

A brick on a tile is illegal

Resting on the ground without a mate is fine. Resting on another part without clutch is not — it would slide. The ghost turns red and tells you why. Enter respects the same rule as a click.

Edits are transactions, not “the mesh moved”

UI code and agent tools both dispatch typed operations such as `part.add`, `part.transform`, `part.recolor`. A successful batch produces exactly one new revision and one undoable transaction. Connection edges are updated as part of that same transaction, so undo removes the clutch the edit created.



Edits are optimistic and strict. A stale plan fails with a repair hint instead of overwriting.

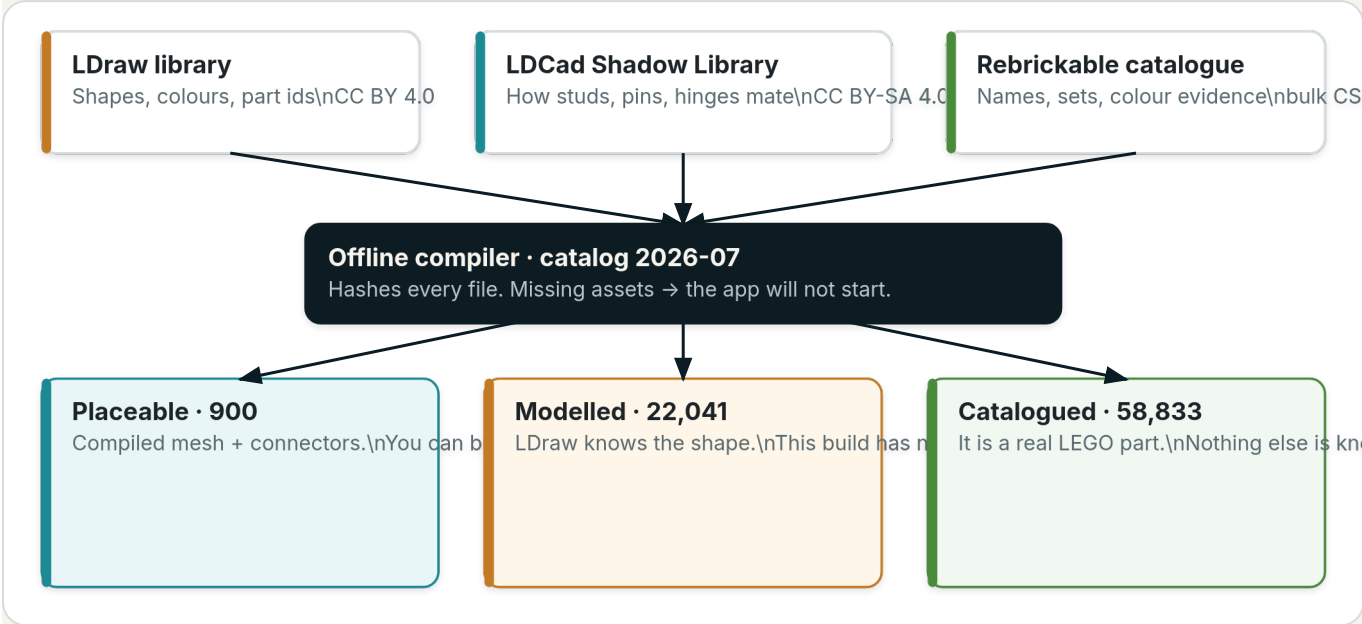
Rules the kernel will not bend

- The revision number only increases — including undo and redo.
- An agent mutation must name the exact revision it read.
- Protected parts and locked subassemblies cannot be silently rewritten.
- Only parts with compiled geometry can be placed. Catalog-only identities return `GEOMETRY_UNAVAILABLE`.
- Preflight (a ghost proposal) does not replace the live document.
- There is no fake catalog. If the compiled assets are missing, the editor refuses to start.

Command deck parity. The same assembly generators a person finds under `ASSEMBLE` are the tools an agent calls. “The human UI can do it but the agent cannot” is treated as a bug, not a phase.

Three levels of knowing a part — and search says which one you got

The library is compiled offline from three independently licensed datasets. At runtime Brickwright does not scrape LEGO’s website and does not invent a mesh when it is missing.



Search always says which tier it found. “Never heard of it” and “real, but not placeable” are different answers.

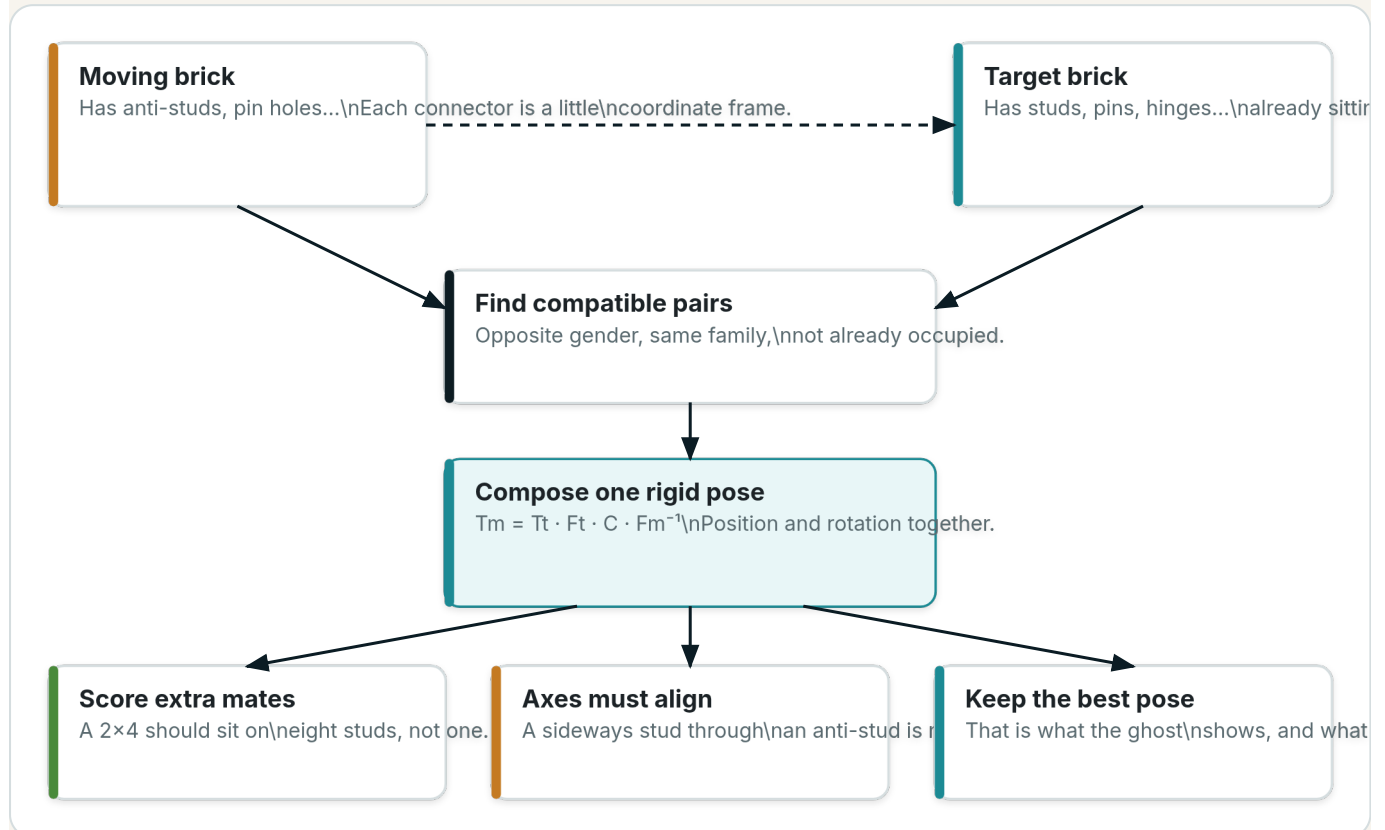
TIER	WHAT IS KNOWN	WHAT YOU CAN DO
Placeable	Mesh, envelope, LDCad connectors	Build, snap, collide, render, export
Modelled	Official LDraw shape and connections; no mesh in this build	Inspect and search. Placing it is refused.
Catalogued	Name, category, set appearances	Confirm it exists. The wider index is fetched only when you search past the modelled library.

Search is ranked, not filtered: an exact part number wins, then a name, then a measured size like 2x4, then a buried word. Official-set frequency breaks ties. Asking the agent to place a search-only part teaches it to call search with `requireGeometry=true`.

Licensing is not the code licence. Brickwright’s source is AGPL-3.0. The catalog bytes are derivatives of LDraw, LDCad, and Rebrickable, and those terms travel with the assets. Attribution is recorded in `licenses.json`.

How two bricks decide they are actually attached

Each placeable part carries connector frames compiled from the LDCad Shadow Library — studs, anti-studs, pins, holes, axles, clips, hinges, balls. A mate is not “close enough in space.” It is two frames brought into coincidence, with aligned axes, and a known leftover freedom (fixed, revolute, cylindrical, spherical, or honestly unknown).



Snapping is frame algebra, not “move until it looks close.” Hinges and Technic pins use the same formula as stacking.

Why the formula looks scary

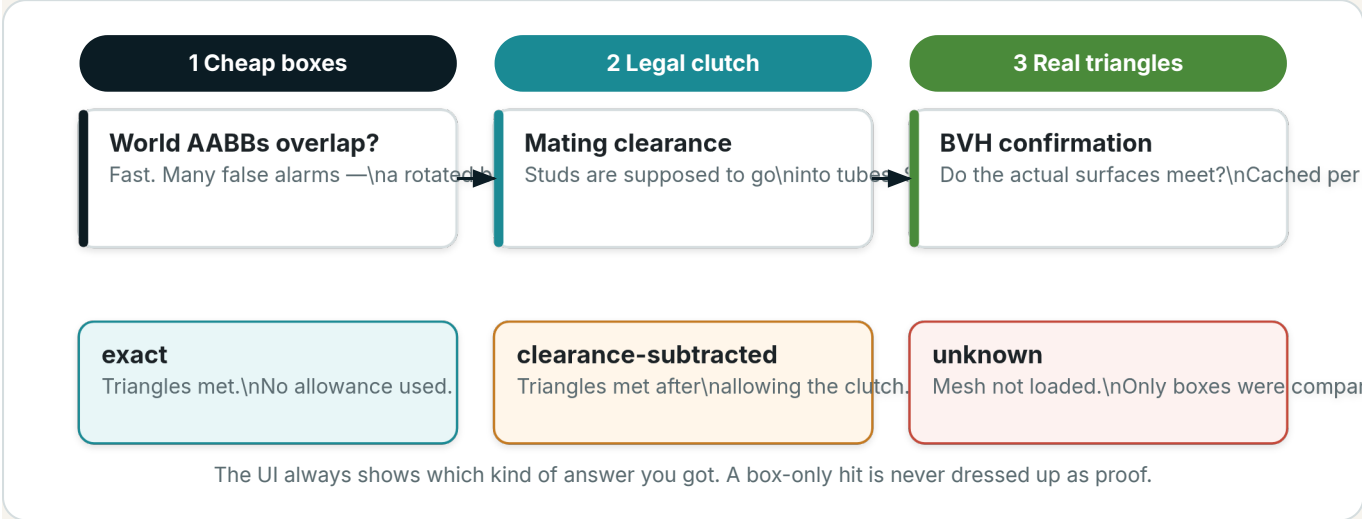
$T_m = T_t \cdot F_t \cdot C \cdot F_m^{-1}$ means: start from the target brick, walk to its connector, apply the joint’s leftover wiggle, then walk back out the moving brick’s connector. You get a full pose. Studs-not-on-top and right-angle Technic fall out of the same expression as ordinary stacking.

The graph is saved

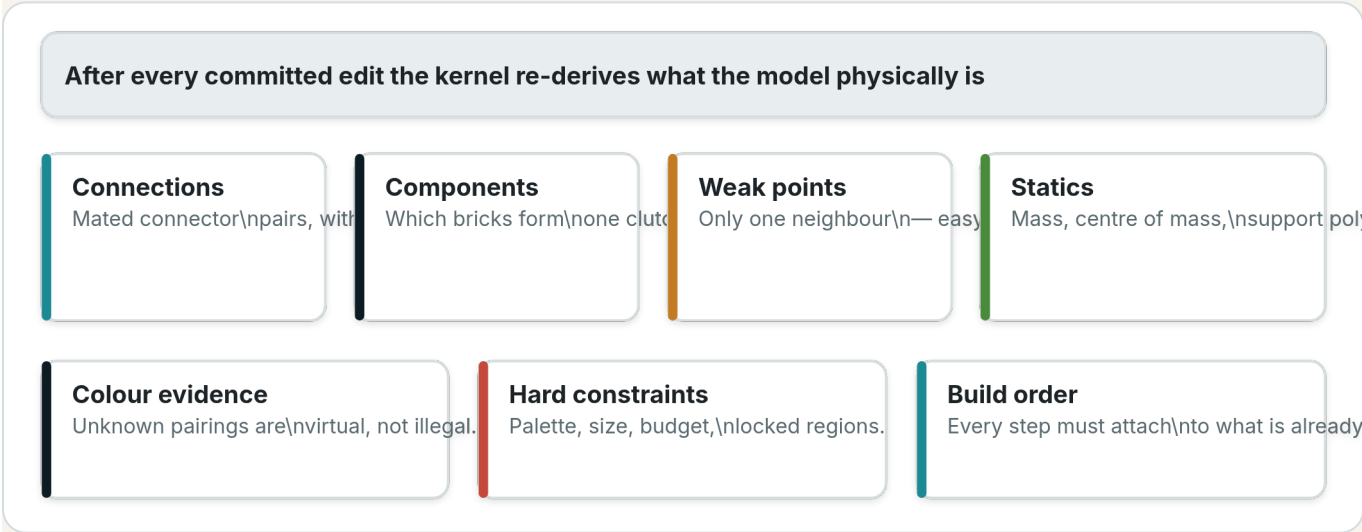
Each mated pair is stored with when it appeared and how it was made (snap, explicit Connect, or inferred on import). That graph is what “is this one piece or three?” and “can I sequence a build guide?” are computed from.

Collision, connectivity, and whether it stands up

A correct stack looks like an intersection in a naive model: the stud occupies the tube. So Brickwright does not shout “collision” at every legal clutch. It also does not pretend a bounding-box overlap is a proven crash.



Collision is honest about its certainty. A stacked brick is not reported as broken just because the studs occupy the tubes.

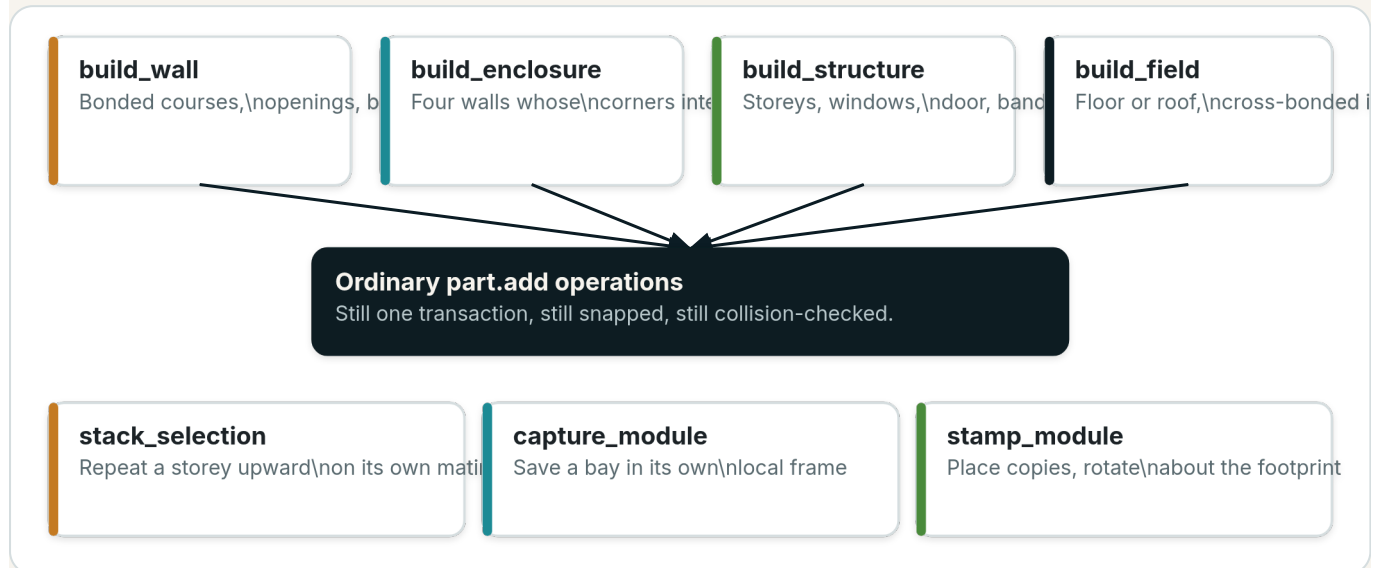


Health is measured from the connection graph and the compiled solids — not from how pretty the render looks.

Mass and tipping come from the compiled surface volume, not from a made-up box. Two caveats are printed on every statics report rather than scaled away: computed mass runs a bit heavy because LDraw is an idealized solid, and clutch strength is an assumption (LEGO publishes none).

One instruction, a whole storey

Placing a wall brick-by-brick is where quality dies: stacked seams, unbonded corners, doors that are just holes. Parametric planners in the kernel do the bricklaying using measured part lengths from this catalog. They emit ordinary `part.add` operations, so undo, ghosts, protection, and collision still apply.



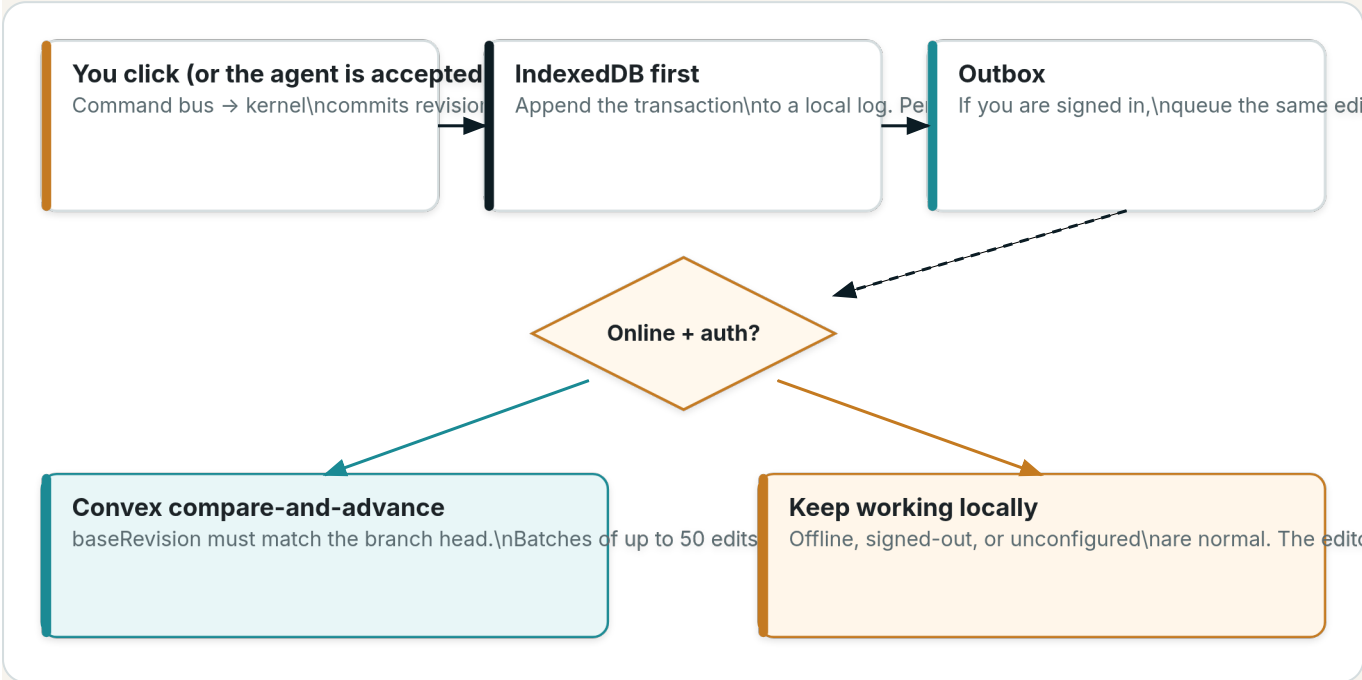
A language model is bad at laying a bonded wall brick by brick. These planners do the bricklaying; the kernel still checks the result.

An opening is not a hole. Where the pack has a window or door that fits, it is seated as a real element and it decides how many courses the opening spans. Courses above and below bridge the edges so a doorway does not become a perforated column you could pull off with your fingers.

You can capture a finished bay as a module and stamp it along a street, rotated about its own footprint. Four such instructions have been measured at over a thousand parts, thousands of mates, and zero collisions — still a handful of undo steps.

Your computer first. The cloud is a replica.

Every committed transaction is appended to an IndexedDB log on top of a periodic checkpoint. Reopening replays forward from the checkpoint. A gap in the log stops replay rather than applying history out of order. The save indicator tells you whether persistence is durable, memory-only, or failing.



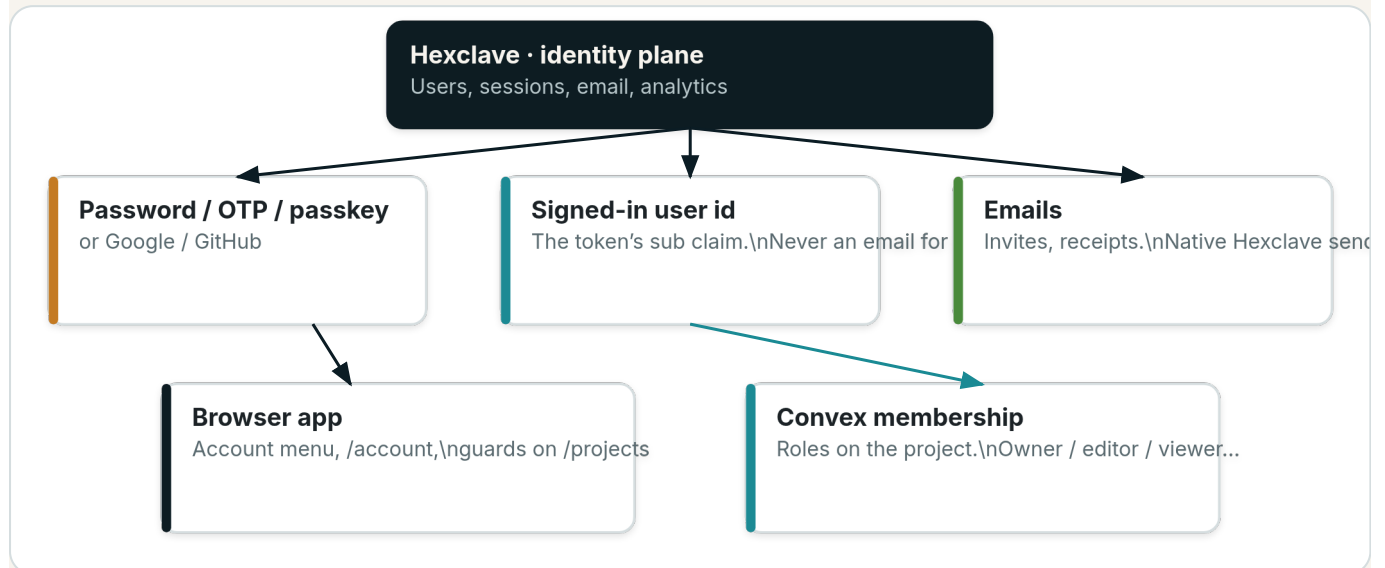
The cloud is a replica, not the original. Your edit is durable on this computer even if the network is gone.

SITUATION	WHAT HAPPENS
Signed out, or Hexclave not configured	Full CAD against local storage. Honest empty account chrome.
Offline while signed in	Edits keep landing in IndexedDB and the outbox. Sync drains later.
Two people edited from the same revision	The second write is STALE_DOCUMENT. Recovery can fork a conflict branch and keep both histories. No last-write-wins.
Upload interrupted	Claims and forks are retry-safe. The same client transaction id is idempotent.

Cloud history is the same log: snapshots plus transactions, paged against a fixed revision, checksummed. A missing page is an error, not a quietly partial model.

Hexclave is who you are. Convex is what you share.

Brickwright uses Hexclave for users, authentication, email, and analytics. Password, one-time codes, passkeys, Google, and GitHub are enabled. The CAD kernel does not know your email. Cloud membership is keyed on the Hexclave user id inside the access token.



Who you are and what the model is are separate planes. Cloud writes still check the Hexclave token.

Invitations

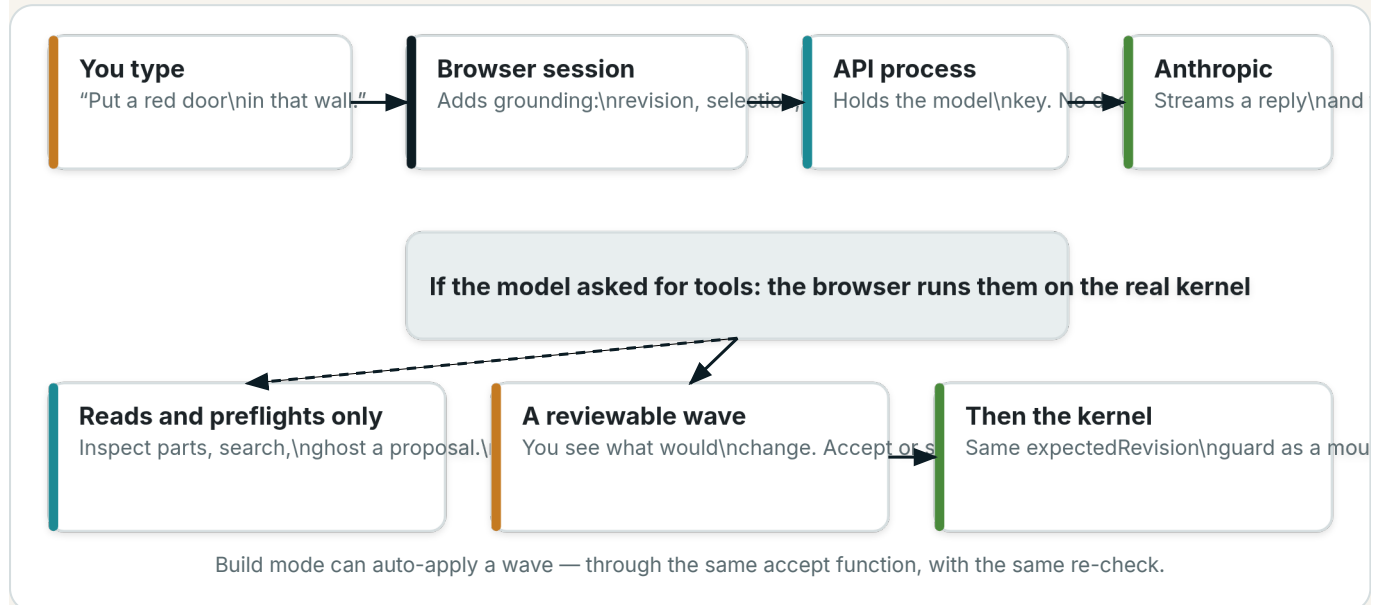
Sharing a project with a person who is not a member yet sends mail through Hexclave. Delivery is retried with a visible status. Expired invites do not block a replacement. Email is an address to deliver to, never an authorisation key.

Analytics are masked

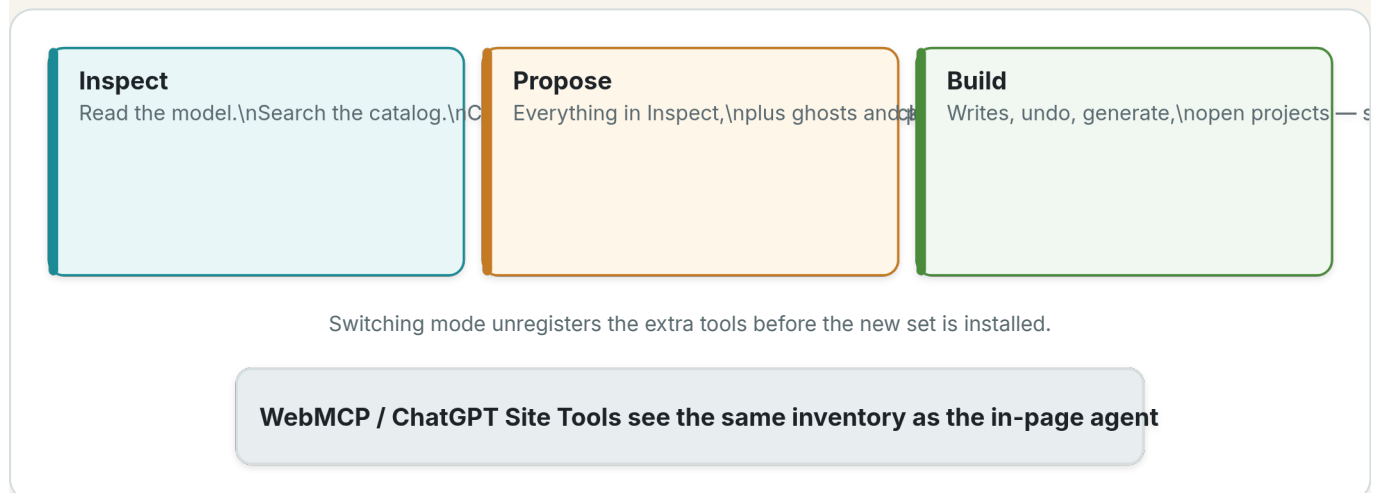
Product analytics fire for shell events. CAD content — part numbers, prompts, model names that would leak a design — is masked so a session replay is not a free copy of the build.

A design partner that cannot sneak a brick in

You talk in ordinary language. The assistant reads the model through tools, plans in the same capability vocabulary as the command deck, and produces **waves** you review. The language model has no commit tool in any mode. The only path onto the command bus is the same accept function a person clicks.



The language model never holds the bricks. Tools run in your browser; the API process only talks to the model.



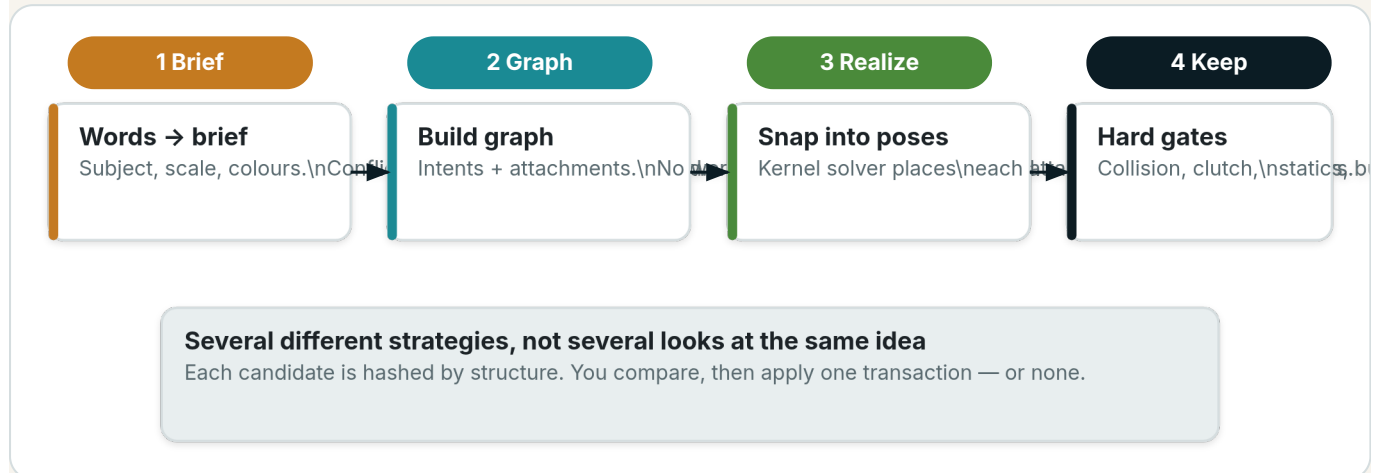
Autonomy is a tool inventory, not a promise. Protected parts and collisions are enforced even in Build mode.

In ChatGPT's desktop browser the same tools register as Site Tools. In an ordinary browser they appear as `window.brickwright.invoke(...)`, which is how tests and developers exercise the identical surface.

Secrets stay off the page. ANTHROPIC_API_KEY lives in the Node API process. The browser sends a grounded transcript to `POST /api/assistant` and receives an NDJSON stream. The server keeps no session: the transcript travels with the request. Timeouts and byte ceilings are enforced there so the page cannot raise them.

From a sentence to bricks that actually clutch

Generation is for “build me a small red fire station,” not for nudging one plate. The important trick: a candidate is never a list of guessed world coordinates. It is a **build graph** of part/region intents joined by connector attachments. The deterministic realiser turns each attachment into a pose with the kernel’s own snap solver. Bulk walls and decks are delegated to the parametric planners.

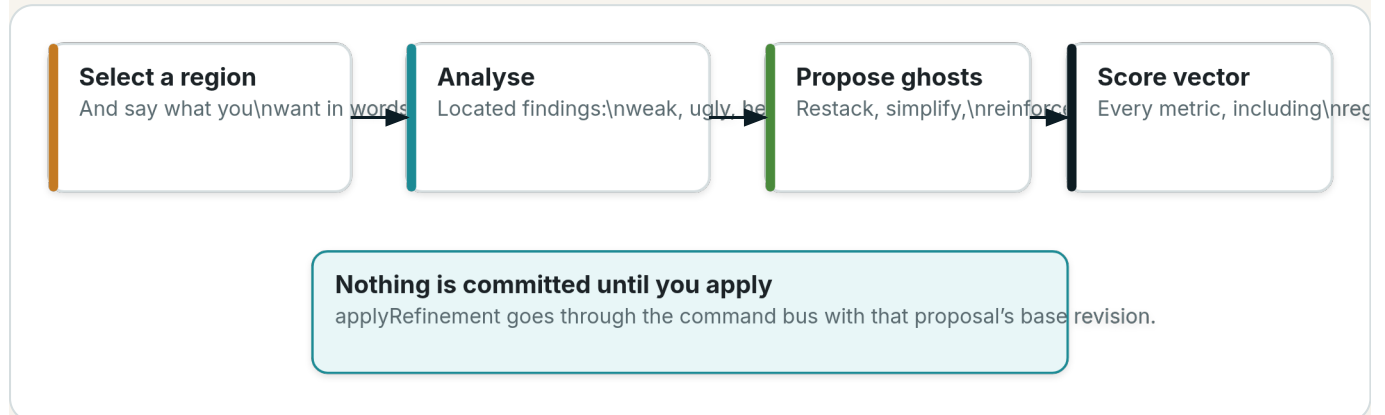


A generated model is not a pile of guessed coordinates. The AI proposes structure; the kernel decides where every brick sits.

Several candidates are produced from different structural strategies and seeds. If they are secretly the same building, the structural hash says so — you do not get three skins of one idea. Applying a candidate is one transaction at the current revision, or a refusal if the document moved.

The design doctor

Refinement is the second half of the loop. Something rough exists. You select a region and say “make the roof lower and cleaner.” The module locates findings, generates alternatives (restack, simplify, reinforce, symmetrize, and friends), scores a full metric vector — regressions included — and hands back ranked ghosts that have already passed the kernel’s checks.

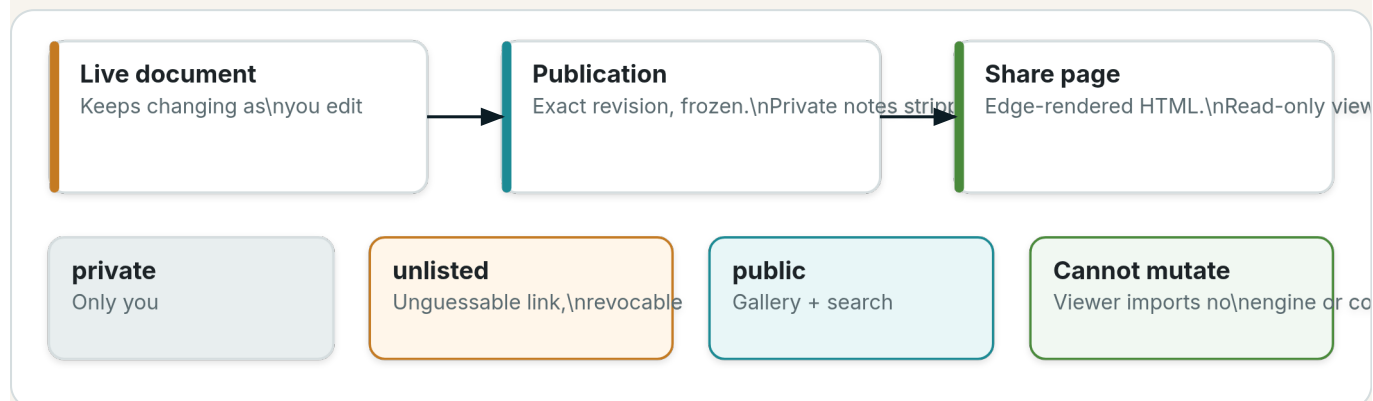


Refinement is a design doctor: it measures a patch of the model and offers ranked, already-checked alternatives.

Heavy work can run in a web worker so the viewport stays alive. A language model, if present, may only set weights and ordering. It still does not get to invent coordinates.

A share link is a photograph of a revision

Publishing captures the document at an exact revision, strips private notes and agent prompts, hashes the bytes, and freezes the object. A second write to the same slug is refused. Open Graph images are rendered from that snapshot at publish time, not fetched from a live GPU later.

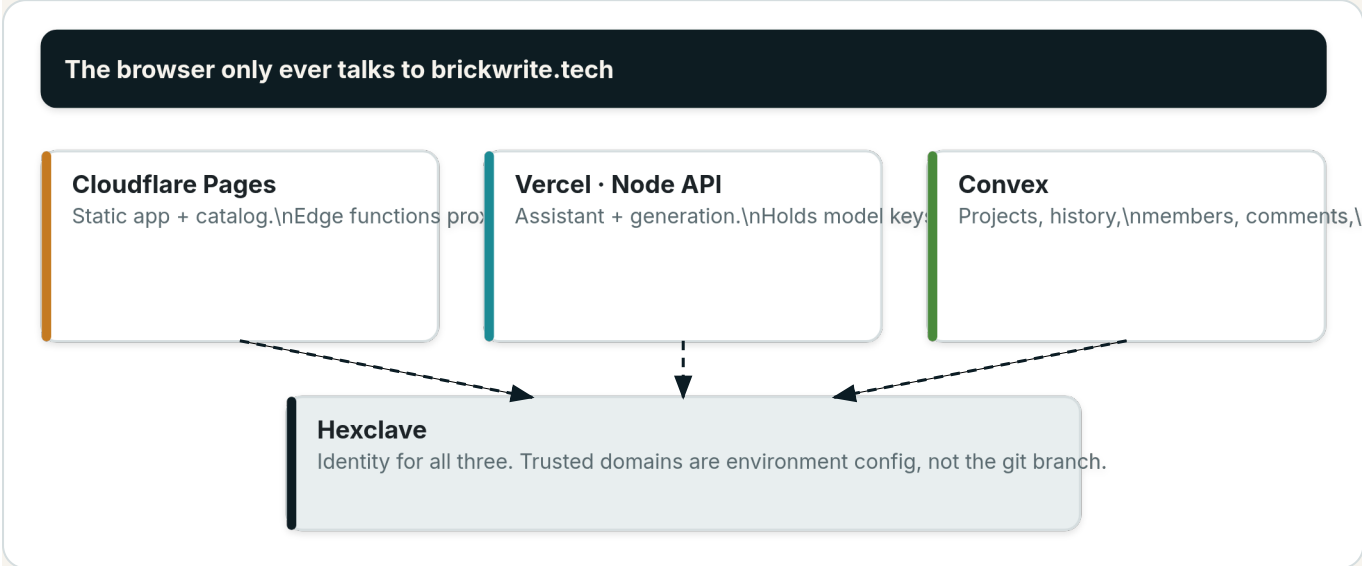


Sharing copies a moment in time. Editing the original afterwards cannot rewrite a published link.

The public viewer is a different program path: it does not import the engine, the session, or the command bus. Forking copies the frozen model into *your* editor as a new document. Export still works where the publication allowed it — .ldr, MPD with submodels, BOM CSV, printable HTML build guides with embedded step pictures.

Three services behind one hostname

Production is `https://brickwrite.tech`. You never talk to the model API's public hostname; an edge proxy rate-limits paid paths and forwards with a shared secret. Unproxied calls to the Vercel origin answer 403 on purpose.



Three services, three kinds of secret. A CAD session without Hexclave or Convex still works — just local, with no account.

SERVICE	HOLDS	IF IT IS MISSING
Cloudflare Pages	The app, catalog, share HTML, API proxy	The site is down.
Vercel Node	Assistant and generation, model keys	CAD still works. Chat and generate fail honestly.
Convex	Project replicas and collaboration	CAD still works locally. Cloud projects unavailable.
Hexclave	Login, email, analytics	CAD still works. Accounts unavailable.

A production build made without the Hexclave project id or Convex URL is a supported mode: a working editor with no account layer. That is also why a misconfigured deploy can look “fine” until you try to sign in.

A map if you want to open the code

You do not need this section to use the product. It is here so the folders match the story above.

<code>src/cad</code>	The kernel: document, snap, collision, assembly, save
<code>src/editor</code>	The cockpit: viewport, panels, gizmos, keyboard
<code>src/webmcp</code>	The tool surface humans and agents share
<code>src/agent + server/assistant</code>	Chat loop; tools in the browser, key on the server
<code>src/generation</code>	Sentence → brief → graph → realised bricks
<code>src/refinement</code>	Design doctor on a selected region
<code>src/intelligence</code>	Smarter part finding than keyword search
<code>src/cloud + convex</code>	Replica, membership, history, invites
<code>src/features</code>	Landing, explore, share, gallery, projects
<code>src/platform + hexclave</code>	Shell, routes, boot, accounts
<code>tools/</code>	Catalog compiler and browser acceptance tests
<code>functions/ + api/</code>	Edge proxy, share HTML, Vercel adapters

Ten workstreams. Each owns a folder. The kernel is imported; it is never rewritten from a feature.

How a change is supposed to travel

1. If it changes what a brick *is*, it belongs in `src/cad` and must go through operations and revisions.
2. If it changes how a brick *looks or is gripped*, it belongs in the editor, which dispatches those operations.
3. If it is a new agent skill, it is a tool schema plus a kernel capability — never a private back door.
4. If it is accounts, email, or analytics, it should use Hexclave rather than a one-off service.

The test that captures the philosophy. A generated building is asserted to be bonded, collision-free, fully billed, sequenced into real steps, and undoable as the number of generator calls you made — not as a screenshot that “looks like a house.”

This guide describes the application as of catalog 2026-07. Numbers (part counts, coverage) are measurements of that committed build. The architecture — one document, two operators, a kernel that refuses to guess — is the part that is meant to stay true as those numbers grow.